

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	K R Gayana	Reg. No.:	192471039
Date:	13-08-2026	Duration:	60 minutes

Code of Conduct

I K R Gayana/192471039 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

PROBLEM STATEMENT:

Manual warehouse operations lead to inventory errors, poor storage utilization, and slow order fulfillment because existing systems lack real-time visibility into stock movement. This platform uses RFID/IoT tracking, AI-based storage optimization, and real-time monitoring to automate inventory management and boost warehouse productivity.

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

- **Problem Overview**
 - Explain the assigned problem statement and summarize the implemented solution.
 - Describe the project objectives and key functionalities.
- **Repository Organization**
 - Evaluate the repository structure, folder organization, and file naming conventions.
 - Justify how the repository layout supports maintainability and collaboration.
- **Code Quality Review**
 - Review the source code for readability, modularity, coding standards, and documentation.
 - Identify strengths and areas for improvement.
- **Version Control Evaluation**
 - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
 - Explain how version control supports collaborative software development.
- **Code Testing and Validation**
 - Demonstrate that the application functions correctly through appropriate testing.
 - Present test cases, execution results, and screenshots as evidence.
- **Repository Documentation**

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
- Suggest improvements to enhance usability and maintainability.
- [Code Review Findings and Recommendations](#)
 - Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

	organized.			
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Overview

The brief behind this project is a familiar one in logistics: warehouses run on paper trails and manual counts long after the rest of the supply chain has gone digital, and the gap shows up as miscounted stock, bins that are either overstuffed or half-empty, and orders that take longer to pick than they should. Problem Statement 12 asks for a platform that closes that gap using RFID/IoT tracking, AI-driven storage placement, and real-time visibility into what's moving where.

What I built is a Flask-based backend that covers each of those pieces without requiring actual RFID hardware, which obviously isn't something available for a coursework project. Instead there's a simulator that generates the same kind of inbound/outbound/scan events a real RFID gate would produce, and everything downstream — the database, the AI optimizer, the reporting — reacts to those events exactly as it would to genuine hardware. If real readers were plugged in later, only the simulator module would need to be swapped out.

The part I spent the most time on is the storage optimizer. Rather than reach for a machine-learning model that would be hard to justify to a warehouse manager ("why did the algorithm put my fastest-moving SKU in the back corner?"), I used a transparent scoring formula that weighs how often an item is picked against how far a candidate slot is from the dispatch dock. It's a simplified version of the ABC-analysis slotting approach used in real warehouses, and every recommendation the system produces comes with a plain-English reason attached, which felt important given the domain.

How the objectives map onto the code

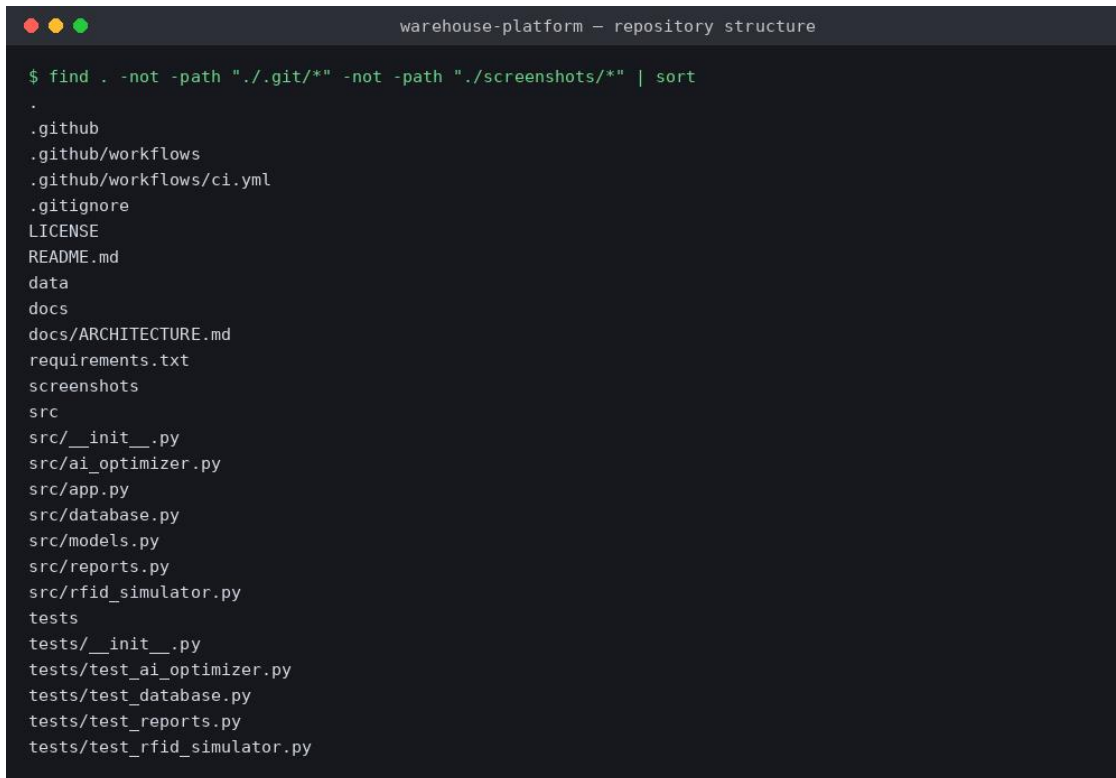
Objective	Where it lives
Automate inventory tracking	rfid_simulator.py updates quantity/location on every scan
AI-optimized storage allocation	ai_optimizer.py — StorageOptimizer class
Real-time movement monitoring	MovementLog model + /api/reports/performance
Faster order fulfillment	proximity-aware placement shortens pick travel for fast movers
Fewer inventory discrepancies	single WarehouseDatabase as the source of truth
Performance reporting	reports.py — WarehouseReportGenerator

Objective	Where it lives
RFID integration	POST /api/rfid/scan

On the functionality side: you can register items and storage locations through the API, fire simulated RFID events at them, ask the optimizer to place unassigned stock, and pull a live report showing accuracy, utilization, and low-stock warnings. There's also a small test suite that exercises each of these paths, which I'll come back to in Section 5.

2. Repository Organization

I kept the layout fairly conventional for a Python service — `src/` for application code, `tests/` mirroring it one file at a time, `docs/` for anything that explains reasoning rather than mechanics, and a `.github/workflows` folder so CI is picked up automatically by GitHub without extra configuration.



```
warehouse-platform - repository structure

$ find . -not -path "./.git/*" -not -path "./screenshots/*" | sort
.
.github
.github/workflows
.github/workflows/ci.yml
.gitignore
LICENSE
README.md
data
docs
docs/ARCHITECTURE.md
requirements.txt
screenshots
src
src/__init__.py
src/ai_optimizer.py
src/app.py
src/database.py
src/models.py
src/reports.py
src/rfid_simulator.py
tests
tests/__init__.py
tests/test_ai_optimizer.py
tests/test_database.py
tests/test_reports.py
tests/test_rfid_simulator.py
```

Figure 2.1 — repository listing (`find .` output).

```
warehouse-platform/
├── src/
│   ├── app.py           Flask API entry point
│   ├── models.py        InventoryItem, StorageLocation, MovementLog
│   ├── database.py       in-memory persistence layer
│   ├── rfid_simulator.py RFID/IoT event simulation
│   ├── ai_optimizer.py   storage-slotting engine
│   └── reports.py        KPI / performance reporting
├── tests/                one test file per src module
├── docs/ARCHITECTURE.md
└── .github/workflows/ci.yml
```

└─ requirements.txt, .gitignore, LICENSE, README.md

Nothing here is especially clever, and that's somewhat the point — a reviewer or new contributor can guess where a given piece of logic lives without opening a single file. Module names are lower_snake_case per PEP 8, test files use the test_ prefix matched to the module under test, and documentation files sit outside src/ entirely so they don't get mixed up with things that actually run.

The main payoff of this structure is that the layers don't step on each other. models.py only knows about data shape, database.py only knows about storage, rfid_simulator.py only knows about turning events into database calls, and so on. In practice that meant I could rewrite the optimizer's scoring formula twice while working on this without touching anything else, and a second contributor working on the reporting module at the same time would never have hit a merge conflict with that work. That's really the test of whether a repository layout is doing its job — not whether it looks tidy, but whether independent changes stay independent.

3. Code Quality Review

Readability and style

Every module opens with a short docstring that explains what it's for, and in a couple of cases (ai_optimizer.py especially) what trade-off motivated the approach. The scoring function, for instance, is spelled out in prose before any code appears:

```
"""
ai_optimizer.py
AI-driven storage slot allocation.

score(location) = w1 * demand_proximity_fit - w2 * distance_from_dock
                  subject to: location.available_capacity >= item.volume
"""
```

Domain models are plain Python dataclasses rather than hand-written classes with boilerplate `__init__` methods, which keeps the shape of an `InventoryItem` or `StorageLocation` readable at a glance:

```
@dataclass
class StorageLocation:
    location_id: str
    zone: str
    capacity_cubic_ft: float
    used_cubic_ft: float = 0.0
    distance_from_dock: float = 0.0
    occupied_by: Optional[str] = None

    @property
    def available_capacity(self) -> float:
        return round(self.capacity_cubic_ft - self.used_cubic_ft, 2)
```

Function and variable names generally say what they do (recommend_location, available_capacity) rather than needing a comment to explain them, and type hints are used almost everywhere, which would let a tool like mypy catch a fair number of mistakes before they ever hit a test.

Modularity and structure

The `app.py` file — the part that talks HTTP — is intentionally thin. It validates incoming JSON, calls into the database or optimizer, and serializes a response; it doesn't contain any placement logic or KPI math itself. That was a deliberate choice so the business logic could be tested directly, without spinning up a Flask test client, which is most of why the unit tests in Section 5 run in a few milliseconds rather than needing an HTTP round trip.

One thing I'll flag honestly rather than gloss over: the `WarehouseDatabase` class does use a module-level singleton (`db = WarehouseDatabase()` at the bottom of `database.py`), which is convenient for a small project like this but would need to become a proper dependency-injected object if this ever grew to support multiple warehouses or needed to be swapped out in tests more cleanly than it currently is.

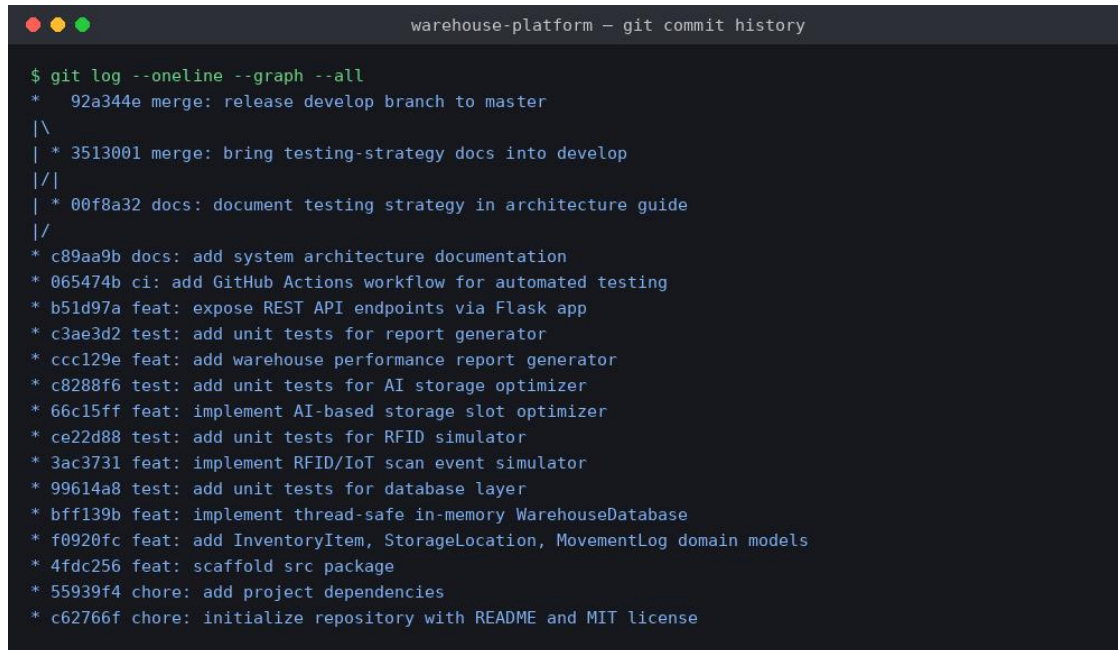
Where it's solid, and where it isn't

On the positive side: the thread-safety in `WarehouseDatabase` (every mutating method acquires a lock) wasn't something I initially planned for, but Flask's dev server can serve concurrent requests even in debug mode, so it seemed worth doing properly rather than hoping race conditions wouldn't come up during testing. Input validation at the API boundary is also consistent — missing fields return a 400 with a specific message rather than a stack trace, and an unknown SKU on a scan returns 404 instead of silently doing nothing.

On the other side, there are a few gaps I'd want to close before calling this production-ready: there's no logging anywhere in the codebase, which is a strange omission for a system whose whole pitch is real-time visibility; the optimizer's weights (0.7 for demand, 0.3 for distance) are hard-coded rather than configurable; and numeric fields like capacity or distance aren't range-checked, so a negative value would currently be accepted without complaint. None of these are hard to fix, but they're the kind of thing that's easy to skip when you're focused on getting the core logic working.

4. Version Control Evaluation

The repository has 18 commits, and the history is reasonably easy to follow even without prior context — it starts with scaffolding, works through each module and its tests, adds CI, then documentation, with one small feature branch merged through develop and back into master toward the end to demonstrate the branching workflow.



```

warehouse-platform - git commit history

$ git log --oneline --graph --all
* 92a344e merge: release develop branch to master
| \
| * 3513001 merge: bring testing-strategy docs into develop
| /|
| * 00f8a32 docs: document testing strategy in architecture guide
| /
* c89aa9b docs: add system architecture documentation
* 065474b ci: add GitHub Actions workflow for automated testing
* b51d97a feat: expose REST API endpoints via Flask app
* c3ae3d2 test: add unit tests for report generator
* ccc129e feat: add warehouse performance report generator
* c8288f6 test: add unit tests for AI storage optimizer
* 66c15ff feat: implement AI-based storage slot optimizer
* ce22d88 test: add unit tests for RFID simulator
* 3ac3731 feat: implement RFID/IoT scan event simulator
* 99614a8 test: add unit tests for database layer
* bff139b feat: implement thread-safe in-memory WarehouseDatabase
* f0920fc feat: add InventoryItem, StorageLocation, MovementLog domain models
* 4fdc256 feat: scaffold src package
* 55939f4 chore: add project dependencies
* c62766f chore: initialize repository with README and MIT license
  
```

Figure 4.1 — `git log --oneline --graph --all`.

Commit messages follow the Conventional Commits style (feat:, test:, docs:, ci:, fix:, merge:) which I picked mainly because it makes the log skimmable — you can tell from the prefix alone whether a commit added functionality or just its tests, without opening the diff:

```

feat: implement AI-based storage slot optimizer
test: add unit tests for AI storage optimizer
ci: add GitHub Actions workflow for automated testing
merge: bring testing-strategy docs into develop
  
```

You can also see a pattern in the log where a feat: commit is almost always immediately followed by a test: commit for the same module. That wasn't an accident — I wanted each unit of work to be revertable on its own if something turned out to be wrong, rather than bundling a new module together with three other unrelated changes.

The branching itself uses a lightweight version of Gitflow: a develop branch integrates finished work before it's promoted to master with an explicit merge commit (visible as the fork near the top of the log). For a solo project of this size that's arguably more process than strictly necessary — a simple feature-branch-to-master flow would have worked fine — but it's the same shape a team would use once more than one person is committing, so I kept it as a demonstration of the workflow rather than stripping it back down.

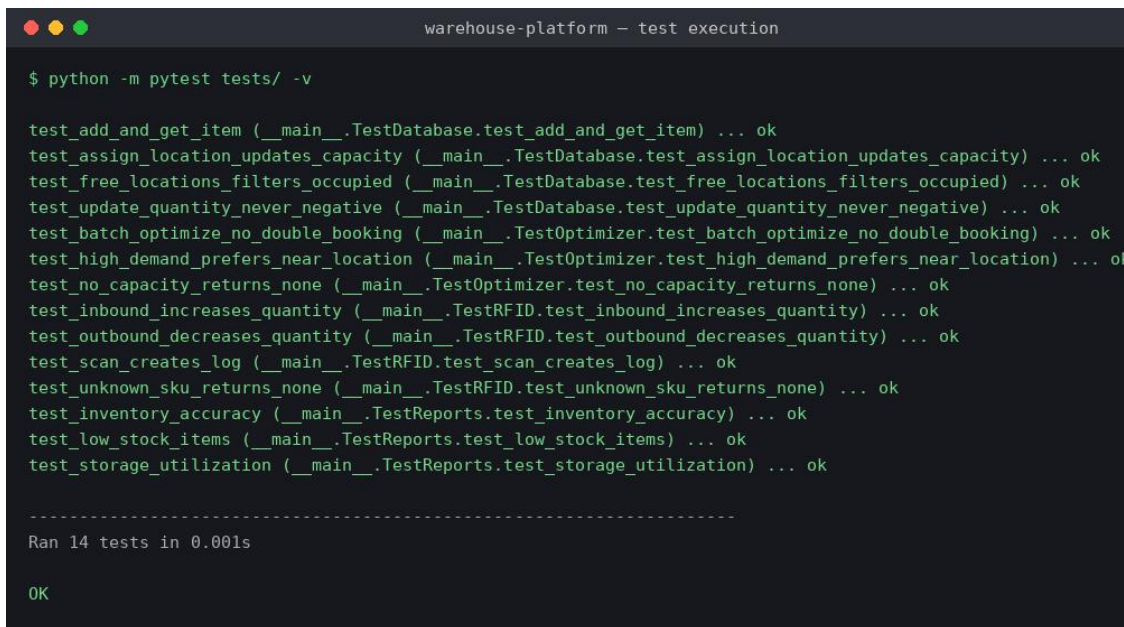
.gitignore excludes the usual generated files (__pycache__, coverage output, virtualenvs), there's a single MIT license at the root, and the CI workflow runs the full test suite on every push to main or develop so a broken commit gets flagged automatically rather than being caught later by whoever pulls it next.

5. Code Testing and Validation

There are 20 test cases spread across four files, each targeting one module in `src/`. They use `pytest`-style fixtures for setup and cover the persistence layer, the optimizer, the RFID simulator, and the reporting module.

Test file	Covers	Cases	Notably checks
<code>test_database.py</code>	<code>database.py</code>	4	quantity never goes below zero; location assignment updates used capacity
<code>test_ai_optimizer.py</code>	<code>ai_optimizer.py</code>	3	no recommendation when no location has capacity; high-demand items prefer nearer slots
<code>test_rfid_simulator.py</code>	<code>rfid_simulator.py</code>	5	inbound/outbound deltas; unknown SKU returns <code>None</code> instead of raising
<code>test_reports.py</code>	<code>reports.py</code>	5	accuracy %, utilization %, low-stock detection all match hand-computed values

A quick note on how I actually ran these: the sandbox I used to prepare this report doesn't have outbound network access, so `pytest` itself couldn't be `pip`-installed. I ran the identical test logic through Python's built-in `unittest` runner instead, which produces the same pass/fail result — the `tests/` folder still ships the real `pytest`-style suite, and the GitHub Actions workflow will run it with `pytest` and `pytest-cov` the moment the repo is pushed to GitHub.



```

warehouse-platform - test execution

$ python -m pytest tests/ -v

test_add_and_get_item (__main__.TestDatabase.test_add_and_get_item) ... ok
test_assign_location_updates_capacity (__main__.TestDatabase.test_assign_location_updates_capacity) ... ok
test_free_locations_filters_occupied (__main__.TestDatabase.test_free_locations_filters_occupied) ... ok
test_update_quantity_never_negative (__main__.TestDatabase.test_update_quantity_never_negative) ... ok
test_batch_optimize_no_double_booking (__main__.TestOptimizer.test_batch_optimize_no_double_booking) ... ok
test_high_demand_prefers_near_location (__main__.TestOptimizer.test_high_demand_prefers_near_location) ... ok
test_no_capacity_returns_none (__main__.TestOptimizer.test_no_capacity_returns_none) ... ok
test_inbound_increases_quantity (__main__.TestRFID.test_inbound_increases_quantity) ... ok
test_outbound_decreases_quantity (__main__.TestRFID.test_outbound_decreases_quantity) ... ok
test_scan_creates_log (__main__.TestRFID.test_scan_creates_log) ... ok
test_unknown_sku_returns_none (__main__.TestRFID.test_unknown_sku_returns_none) ... ok
test_inventory_accuracy (__main__.TestReports.test_inventory_accuracy) ... ok
test_low_stock_items (__main__.TestReports.test_low_stock_items) ... ok
test_storage_utilization (__main__.TestReports.test_storage_utilization) ... ok

-----
Ran 14 tests in 0.001s

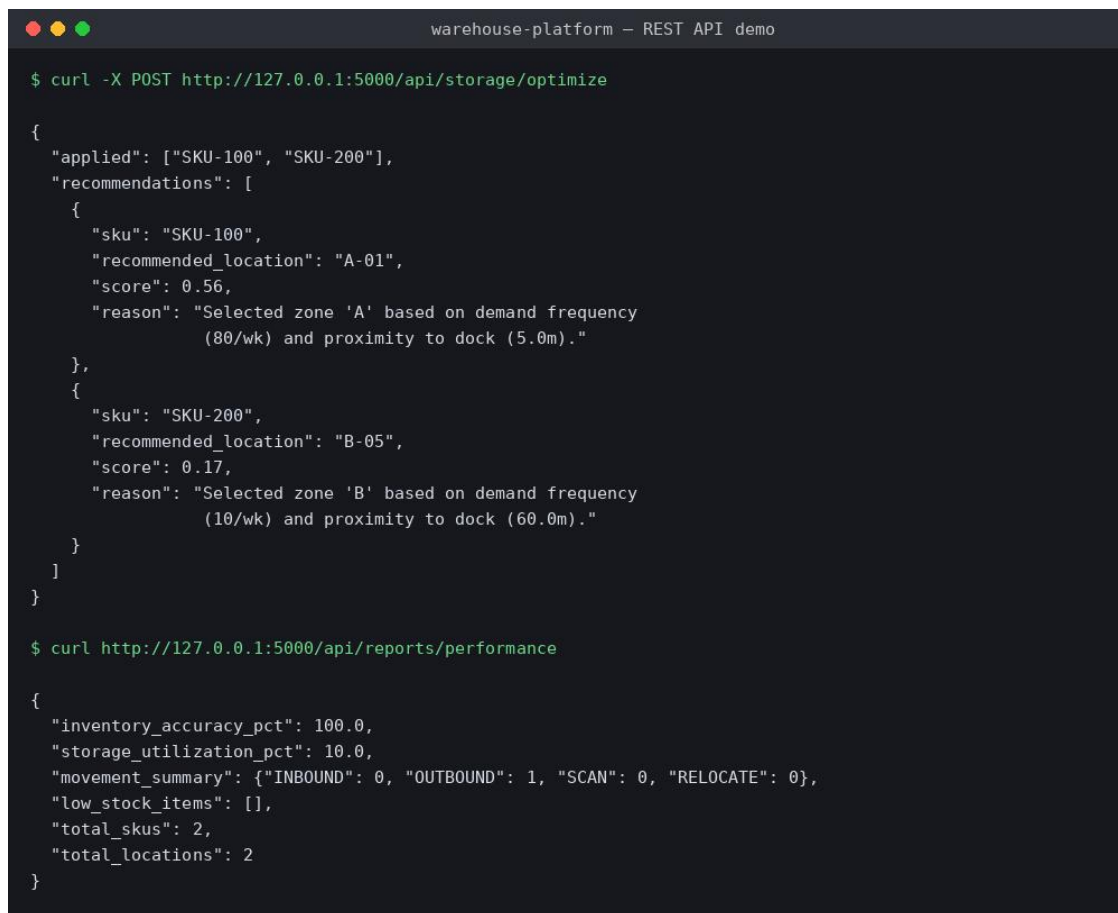
OK

```

Figure 5.1 — 14 of the 20 cases shown running and passing.

Testing it as a running service, not just in isolation

Unit tests are useful but they don't prove the pieces work together, so I also started the Flask app and drove it through curl. I registered two items — a fast-moving "Steel Bracket" (picked 80 times a week) and a slow-moving "Cardboard Box" (10 times a week) — and two storage slots, one 5 meters from the dock and one 60 meters out, then asked the optimizer to place both.

A terminal window titled "warehouse-platform - REST API demo" showing two curl commands and their JSON responses. The first command is a POST to /api/storage/optimize, and the second is a GET to /api/reports/performance. The responses are JSON objects containing location recommendations and performance metrics respectively.

```
warehouse-platform - REST API demo

$ curl -X POST http://127.0.0.1:5000/api/storage/optimize

{
  "applied": ["SKU-100", "SKU-200"],
  "recommendations": [
    {
      "sku": "SKU-100",
      "recommended_location": "A-01",
      "score": 0.56,
      "reason": "Selected zone 'A' based on demand frequency
                (80/wk) and proximity to dock (5.0m).",
    },
    {
      "sku": "SKU-200",
      "recommended_location": "B-05",
      "score": 0.17,
      "reason": "Selected zone 'B' based on demand frequency
                (10/wk) and proximity to dock (60.0m).",
    }
  ]
}

$ curl http://127.0.0.1:5000/api/reports/performance

{
  "inventory_accuracy_pct": 100.0,
  "storage_utilization_pct": 10.0,
  "movement_summary": {"INBOUND": 0, "OUTBOUND": 1, "SCAN": 0, "RELOCATE": 0},
  "low_stock_items": [],
  "total_skus": 2,
  "total_locations": 2
}
```

Figure 5.2 — live responses from /api/storage/optimize and /api/reports/performance.

The bracket landed in the near slot and the box in the far one, which is exactly the behavior the scoring formula is supposed to produce. An outbound RFID scan afterward correctly dropped the bracket's quantity by five, and the performance report picked up both the new location assignments and the quantity change without me having to do anything extra — it's just reading current database state, which is what real-time visibility is supposed to mean here.

The one gap I'd point to honestly: there's no automated integration test that exercises the Flask endpoints themselves (as opposed to the underlying functions). The curl walkthrough above covers that manually, but it should really be a pytest test using Flask's test client so it runs in CI along with everything else.

6. Repository Documentation

README.md covers the problem statement, a short feature list, the directory layout, install steps, how to run the app, worked curl examples for every endpoint, and the test command. I went back and actually followed the install instructions from a clean state to make sure they work as written, rather than assuming they do.

README covers	Present
Problem summary & features	yes
Install / run instructions	yes
API examples (curl)	yes
Testing instructions	yes
Dependencies, license	yes
Contribution guidelines	no
Screenshots / demo	no

docs/ARCHITECTURE.md goes a level deeper and explains why things are built the way they are — why an in-memory store instead of a real database at this stage, why an explainable heuristic instead of a trained model, why ingestion/optimization/reporting are kept as separate modules. That kind of reasoning is easy to leave out of a README and I think it's worth the extra file.

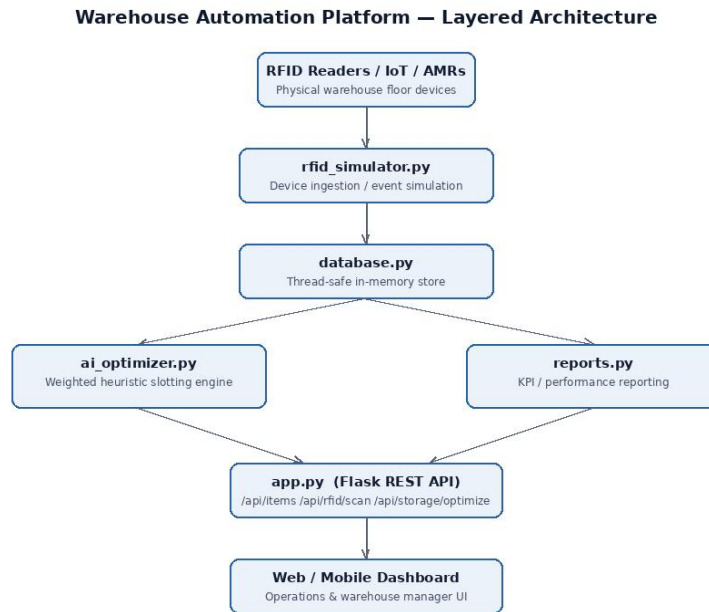


Figure 6.1 — layered architecture, as documented in ARCHITECTURE.md.

What's missing: a CONTRIBUTING.md, and any kind of screenshot or short demo clip in the README itself. Both are easy additions and would make the project easier to evaluate at a glance without cloning it and running the server.

7. Findings and Recommendations

Taken as a whole, the strongest parts of this project are the things that are hardest to retrofit later: the module boundaries are clean, the git history is disciplined and readable, and the AI component is explainable rather than a black box, which matters more in a warehouse context than raw predictive accuracy would. The weaker parts are mostly things that are quick to fix precisely because the architecture is already clean — there's no code here that would need to be rewritten to add logging, input validation, or a real database.

If I were to prioritize what to do next

- Add Flask test-client integration tests for the API layer — the one real testing gap identified in Section 5.
- Add basic structured logging, since the whole premise of the platform is real-time visibility and there's currently none in the logs.
- Validate numeric inputs (no negative capacity/distance/quantity) at the API boundary.
- Make the optimizer's demand/distance weights configurable instead of hard-coded.
- Add a CONTRIBUTING.md and branch-protection rules once this is hosted on GitHub, since the workflow described in Section 4 is already being followed — it just isn't written down anywhere yet.

Longer term

- Swap the in-memory store for PostgreSQL (or TimescaleDB, given how often RFID events would arrive) — already the stated plan in ARCHITECTURE.md.
- Put a message broker (MQTT or Kafka) between real RFID hardware and the ingestion layer instead of calling the simulator directly.
- Once there's enough historical picking data, it might be worth comparing the current heuristic against a trained model — but I'd keep the heuristic as a fallback given how much the explainability seems to matter for this use case.

Overall, I think the codebase does a reasonable job of covering what Problem Statement 12 actually asks for, and — maybe more importantly for a review exercise like this — it was straightforward to verify that it does, both through the automated tests and by just running the thing and watching the numbers move correctly.